Using mefpp:
the WFI preprocessing engine

(fselman@eso.org)

eclipse: 5.0.0
qfits: 5.4.0
mefpp: 1.2.0

Introduction

The mefpp pre-processing engine is written in C and based on the Eclipse and qfits software package. It modifies extensively the Eclipse wfi package. It is still at a very early stage of development and should be used carefully. In this document I give an example reduction session so that a user can get started quickly. Currently the only way to change reduction parameters is by recompiling the code. This is done by choice as the author feels that at this stage only people with programming skills can use it without supervision.

The main characteristic of this package is that it maintains the integrity of the Multi Extension FITS files without splitting them into their 8 extensions. Given the complexity of data management when “reducing” astronomical data it was felt that a factor of 8 in the number of files is too high a price to pay.

The program can be used to create calibration files and then reduce the data in one go, or to first produce the calibration data, and only then reduce the science data with the calibrations previously determined. We will follow the last approach in this tutorial.

The current version is based in eclipse-5.0.0, qfits-5.4.0, amd mefpp-1.2.0.

Installation instructions

The program has to be compiled in three sequential stages. Although the main directory created after un-tarring the file is called mefpp-1.0, it really is the top directory of eclipse-4.9.0. Inside this directory there two subdirectories of interest: qfits, and ins/mef.

- Compile qfits:
  - cd into ${MEFPP}/qfits and run ./configure –prefix=${MEFPP}
  - then type: make;
  - when the compilation ends, type: make install
- Compile eclipse:
  - cd ${MEFPP}
  - run ./configure –prefix=${MEFPP}
  - run make;
  - run make install
  - run: e_setup
- Compile mefpp:
  - cd ${MEFPP}/ins/mef
  - run make

At runtime one could get an error about maximum number of pointer allocated. If that happens I suggest to edit the file xmemory.c en qfits/src and change the line:

```
#define XMEMORY_MAXPTRS    200000
```

to a larger value (multiple of 2).

Quick summary for the impatient

Step 0: Explore data with dfits and fitsort
Step 1: Partition data by creating ascii lists using dfits and fitsort
Step 2: Produce calibrations:

```
mefpp --bias bias.list --biassave --dome domeff_Rc.list --domesave \
--skyff skyff_RC.list --skysave
```

which outputs into working dir:

WFI_MB_1x1.fits
WFI_DOMEFF_BB_Rc_162_ESO844_1x1.fits
WFI_SKYFF_BB_Rc_162_ESO844_1x1.fits
WFI_MFF_BB_Rc_162_ESO844_1x1.fits

Step 3: reduce science frames listed in science.list

```
mefpp --bias WFI_MB_1x1.fits --mff WFI_SKYFF_BB_Rc_162_ESO844_1x1.fits \
--science science.list
```

Step 4: **split the MEF files into 8 extention files each and create at the same time a list with the names of these files, imageNames.list, and an ascii file with the appropriate offset, imageOffset.list.**

```
splitChips toBeSplit.list
```

Step 5: **Using IRAF imcombine to create the mosaic**

Manual Page

mefpp: This program accepts the following switches

--bias <bias.list> or bias.fits

where <bias.list> is a list of Dark frames to be used to calculate a bias frame. You can also give the name of a previously determined bias fits frame, bias.fits.
If <bias.list> is given the calculated frame will not be saved unless –biassave is used.

--biassave

if this switch is present it indicates to mefpp to store the bias frame i(f bias.list is given)

--bpm <bpm.list>

where <bpm.list is list of FITS files to be used to calculate a bad pixel mask. These are usually dome flats with different intensities that will be used in a regression analysis pixel by pixel. If omitted the mefpp pre-processing engine requires that you have the BPM files in the working directory: WFI_BPM_01.fits, ..., WFI_BPM_08.fits.

--dome <dome.list> or dome.fits

<dome.list> or dome flat field fits frame to be used in the pre-processing. If <dome.list> is given the fits files contained in the list will be used to calculate a dome flat field frame.

--domesave

include this switch to save the calculated dome flat field, otherwise it will only be used during the mefpp run and will be lost at the end of the processing

--sky <sky.list> or sky.fits

<sky.list> of sky flat field frame to be used in the pre-processing.
If <sky.list> is given the FITS files contained in the list will be used to calculate the sky flat field frame.

--skysave

include this switch to save the sky file (important for WFI frames)

--mff mff.fits

master flat field frame name to store the resulting frame of combining sky and dome flats, or if a fits file with this name already exist in disk use it for the pre-processing. This last case is what we use at WFI were we specify the previously saved sky flat field as our reduction master flat field.

--science <science.list>

<science.list> is a file containing the science FITS files that will be pre-processed.

Step-by-step guide

We will assume that you have several nights of data in disk some directory that we will call **rawdir**. Let us assume that the data are store at /data/raw: Our **rawdir** can be any of

/data/raw/2005-07-03 /data/raw/2005-07-04 /data/raw/2005-07-05

We will also assume that you have a directory where you will carry out the reductions, the **workdir**. In what follow it will assumed that you will be issuing commands from a terminal from **workdir.**

Step 0: Explore the data with **dfits** and **fitsort**. To explore the data it is usefule to be aware that the image files from WFI are stored in Multi-Extension FITS format files, henceforth MEF files. Each of these files consists in a primary header without any image data associated to it, and a sequence of 8 extensions with their own headers and data. Each FITS header consist of a set of 80-character lines with the following format:

KEYWORD = VALUE / Comments

The following is an excerpt of a primary header of a WFI MEF file:

```
SIMPLE  =                    T         / Standard FITS format (NOST-100.0)
BITPIX  =                  -32 / Bits per pixel
NAXIS   =                    0         / # of axes in frame
EXTEND  =                    T / Extension may be present.
ORIGIN  = 'ESO     '                   / European Southern Observatory
DATE    = '2005-07-05T01:45:28.053'     / UT date when this file was written
MJD-OBS =       53556.06976567         / MJD start (2005-07-05T01:40:27.754)
DATE-OBS= '2005-07-05T01:40:27.754'     / Date of observation
EXPTIME =            299.9178         / Total integration time
OBJECT  = '9PTempel1'                   / Target description
INSTRUME= 'WFI     '                  / Instrument used
OBSERVER= 'UNKNOWN '                    / Name of observer
PI-COI  = 'UNKNOWN '                    / Name(s) of proposer(s)
TELESCOP= 'MPI-2.2 '                    / ESO Telescope Name
RA      =          204.913936         / 13:39:39.3 RA (J2000) at start in (deg
DEC     =           -9.99834          / -09:59:54.0 DEC (J2000) at start in (d
EQUINOX =               2000.         / Standard FK5 (years)
RADECSYS= 'FK5     '                    / Coordinate reference frame
LST     =          57000.763          / 15:50:00.763 LST at start (sec)
UTC     =           6018.541          / 01:40:18.541 UTC at start (sec)
......
```

Of all the keywords available in a FITS file the following are the one useful for the reduction of WFI data:

```
OBJECT
EXPTIME
RA
DEC
HIERARCH ESO TEL AIRM START
HIERARCH ESO INS FILT1 NAME
```

where we can see the infamous hierarchical ESO keywords.

Data exploration in the context we are talking about refer to the visualization of the files in the **rawdir** to search for the calibration files and for the science files and to produce ascii lists with selected partitions, e.g. All the deep R band images of the Tempel1 comet. For this we use de Villard's Eclipse command **dfits** and **fitsort**.

```
$dfits /data/reduced/WFI/DeepImpact/2005-07-03/WFI*fits | \
 fitsort OBJECT EXPTIME INS.FILT1.NAME
```

where the backslash character indicate that the command continues in the next line.

An excerpt of the output is displayed below:

```
/data/raw/2005-07-03/WFI.2005-07-03T22:05:25.581.fits  FLAT,SKY    0.8540     MB#445/18_ESO873
/data/raw/2005-07-03/WFI.2005-07-03T22:06:23.775.fits  FLAT,SKY    1.0587     MB#445/18_ESO873
/data/raw/2005-07-03/WFI.2005-07-03T22:07:23.709.fits  FLAT,SKY    1.3219     MB#445/18_ESO873
/data/raw/2005-07-03/WFI.2005-07-03T22:08:24.084.fits  FLAT,SKY    1.6523     MB#445/18_ESO873
/data/raw/2005-07-03/WFI.2005-07-03T22:09:24.058.fits  FLAT,SKY    2.0543     MB#445/18_ESO873
/data/raw/2005-07-03/WFI.2005-07-03T22:12:20.032.fits  OTHER       1.0000     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-03T22:13:36.057.fits  FLAT,SKY    0.7049     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-03T22:14:33.161.fits  FLAT,SKY    0.8666     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-03T22:15:34.236.fits  FLAT,SKY    1.0865     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-03T22:16:31.801.fits  FLAT,SKY    1.3463     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-03T22:17:34.085.fits  FLAT,SKY    1.6953     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-03T22:20:07.346.fits  OTHER       0.9996     BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-03T22:20:57.610.fits  FLAT,SKY    3.5398     BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-03T22:21:59.016.fits  FLAT,SKY    4.4431     BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-03T22:23:04.190.fits  FLAT,SKY    5.6647     BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-03T22:24:08.914.fits  FLAT,SKY    7.2247     BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-03T22:25:14.649.fits  FLAT,SKY    9.2207     BB#Rc/162_ESO844
.........
/data/raw/2005-07-03/WFI.2005-07-04T11:00:29.768.fits  OTHER       1.9919     BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-04T11:04:05.984.fits  OTHER       1.9915     BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-04T11:04:54.708.fits  FLAT,SKY    38.0708    BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-04T11:06:36.506.fits  FLAT,SKY    26.7761    BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-04T11:08:13.443.fits  FLAT,SKY    18.9175    BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-04T11:09:43.019.fits  FLAT,SKY    13.7463    BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-04T11:11:07.746.fits  FLAT,SKY    10.1079    BB#Rc/162_ESO844
/data/raw/2005-07-03/WFI.2005-07-04T11:13:15.695.fits  OTHER       1.9913     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-04T11:14:04.569.fits  FLAT,SKY    3.5191     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-04T11:15:12.904.fits  FLAT,SKY    2.7394     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-04T11:16:22.989.fits  FLAT,SKY    2.1155     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-04T11:17:33.535.fits  FLAT,SKY    1.6355     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-04T11:18:48.560.fits  FLAT,SKY    1.2408     BB#B/123_ESO878
/data/raw/2005-07-03/WFI.2005-07-04T11:21:24.412.fits  OTHER       1.9800     MB#721/25_ESO847
/data/raw/2005-07-03/WFI.2005-07-04T11:22:43.027.fits  FLAT,SKY    2.3621     MB#721/25_ESO847
/data/raw/2005-07-03/WFI.2005-07-04T11:23:46.762.fits  FLAT,SKY    1.8759     MB#721/25_ESO847
/data/raw/2005-07-03/WFI.2005-07-04T11:24:58.457.fits  FLAT,SKY    1.4379     MB#721/25_ESO847
/data/raw/2005-07-03/WFI.2005-07-04T11:26:08.433.fits  FLAT,SKY    1.1108     MB#721/25_ESO847
/data/raw/2005-07-03/WFI.2005-07-04T11:27:23.448.fits  FLAT,SKY    0.8416     MB#721/25_ESO847
/data/raw/2005-07-03/WFI.2005-07-04T11:28:53.815.fits  OTHER       1.9916     MB#445/18_ESO873
/data/raw/2005-07-03/WFI.2005-07-04T11:29:43.239.fits  FLAT,SKY    0.6497     MB#445/18_ESO873
/data/raw/2005-07-03/WFI.2005-07-04T11:30:53.904.fits  FLAT,SKY    0.5014     MB#445/18_ESO873
```

One can then use the above command together with the unix **grep** command to select appropriate files, e.g.

```
$dfits /data/reduced/WFI/DeepImpact/2005-07-03/WFI*fits | fitsort OBJECT EXPTIME  \
 INS.FILT1.NAME | grep FLAT,SKY | grep Rc | awk '{printf "%s\n", $1}' >ffsky_Rc.list
```

$cat ffsky_Rc.list

/data/raw/2005-07-03/WFI.2005-07-03T22:20:57.610.fits
/data/raw/2005-07-03/WFI.2005-07-03T22:21:59.016.fits
/data/raw/2005-07-03/WFI.2005-07-03T22:23:04.190.fits
/data/raw/2005-07-03/WFI.2005-07-03T22:24:08.914.fits
/data/raw/2005-07-03/WFI.2005-07-03T22:25:14.649.fits
/data/raw/2005-07-03/WFI.2005-07-04T11:04:54.708.fits
/data/raw/2005-07-03/WFI.2005-07-04T11:06:36.506.fits
/data/raw/2005-07-03/WFI.2005-07-04T11:08:13.443.fits
/data/raw/2005-07-03/WFI.2005-07-04T11:09:43.019.fits

/data/raw/2005-07-03/WFI.2005-07-04T11:11:07.746.fits

Step 1: Create ascii files with the bias frames, bad pixel map frames, dome flat frames, and sky flat frames:

$cat bias.list
/data/raw/2005-07-03/WFI_Dark.1.fits
/data/raw/2005-07-03/WFI_Dark.2.fits
/data/raw/2005-07-03/WFI_Dark.3.fits
/data/raw/2005-07-03/WFI_Dark.4.fits
/data/raw/2005-07-03/WFI_Dark.5.fits
/data/raw/2005-07-03/WFI_Dark.6.fits
/data/raw/2005-07-03/WFI_Dark.7.fits
/data/raw/2005-07-03/WFI_Dark.8.fits
/data/raw/2005-07-03/WFI_Dark.9.fits

$cat bpm.list
/data/raw/2005-07-03/WFI_FlatLampIma.fits
/data/raw/2005-07-03/WFI_FlatLampIma.1.fits
/data/raw/2005-07-03/WFI_FlatLampIma.2.fits
/data/raw/2005-07-03/WFI_FlatLampIma.3.fits
/data/raw/2005-07-03/WFI_FlatLampIma.4.fits
/data/raw/2005-07-03/WFI_FlatLampIma.5.fits
/data/raw/2005-07-03/WFI_FlatLampIma.6.fits
/data/raw/2005-07-03/WFI_FlatLampIma.7.fits

$cat domeff_Rc.list
/data/raw/2005-07-03/WFI_FlatDomeIma.fits
/data/raw/2005-07-03/WFI_FlatDomeIma.1.fits
/data/raw/2005-07-03/WFI_FlatDomeIma.2.fits
/data/raw/2005-07-05/WFI_FlatDomeIma.6.fits
/data/raw/2005-07-05/WFI_FlatDomeIma.7.fits
/data/raw/2005-07-05/WFI_FlatDomeIma.8.fits

$cat skyff_Rc.list
/data/raw/2005-07-03/WFI_FlatSkyIma.fits
/data/raw/2005-07-03/WFI_FlatSkyIma.1.fits
/data/raw/2005-07-03/WFI_FlatSkyIma.2.fits
/data/raw/2005-07-03/WFI_FlatSkyIma.3.fits
/data/raw/2005-07-03/WFI_FlatSkyIma.4.fits

In the above example we have provided a list with raw frames for all types of calibrations. In practice the user will probably skip the bad pixel masks which can be obtained from the WFI web pages. They come in single extension FITS files:

WFI_BPM_01.fits
WFI_BPM_02.fits
WFI_BPM_03.fits
WFI_BPM_04.fits
WFI_BPM_05.fits
WFI_BPM_06.fits
WFI_BPM_07.fits
WFI_BPM_08.fits

They have one byte per pixel and have the value 1 if the pixel is OK and 0 if it is bad. ***If the user prefer not to create the BPM files him/herself then the above files should be present in the directory where the data is being reduced.***

Step 2: Produce the calibration data the user should execute the command:

```
$mefpp --bias bias.list --biassave --dome domeff_Rc.list --domesave \
    --skyff skyff_Rc.list --skysave
```

At the end of this command, which can take a long time to execute, the program will have written to disk the files:

WFI_MB_1x1.fits
WFI_DOMEFF_BB_Rc_162_ESO844_1x1.fits
WFI_SKYFF_BB_Rc_162_ESO844_1x1.fits
WFI_MFF_BB_Rc_162_ESO844_1x1.fits

These are MEF files in floating point with a size of 268MBytes each.

Step 3: Reduce the science frames by writing their names in a file, e.g. tempel1_Rc.list

```
$dfits  /data/raw/2005-07-03/WFI*fits | fitsort OBJECT EXPTIME INS.FILT1.NAME |  \
  grep Tempel1 | grep Rc | grep 299 | awk '{printf "%s\n",$1}' >Tempel1_Rc.list
$cat tempel1_Rc.list
/data/raw/2005-07-03/WFI.2005-07-03T23:07:50.699.fits
/data/raw/2005-07-03/WFI.2005-07-03T23:16:54.380.fits
/data/raw/2005-07-03/WFI.2005-07-03T23:23:58.061.fits
/data/raw/2005-07-03/WFI.2005-07-03T23:38:16.955.fits
/data/raw/2005-07-03/WFI.2005-07-03T23:45:17.787.fits
/data/raw/2005-07-03/WFI.2005-07-03T23:52:22.338.fits
/data/raw/2005-07-03/WFI.2005-07-04T00:01:44.400.fits
/data/raw/2005-07-03/WFI.2005-07-04T00:09:40.275.fits
/data/raw/2005-07-03/WFI.2005-07-04T00:17:19.940.fits
/data/raw/2005-07-03/WFI.2005-07-04T00:24:33.131.fits
/data/raw/2005-07-03/WFI.2005-07-04T00:38:49.766.fits
/data/raw/2005-07-03/WFI.2005-07-04T00:47:10.873.fits
/data/raw/2005-07-03/WFI.2005-07-04T00:54:27.265.fits
/data/raw/2005-07-03/WFI.2005-07-04T01:12:37.306.fits
/data/raw/2005-07-03/WFI.2005-07-04T01:19:45.878.fits
/data/raw/2005-07-03/WFI.2005-07-04T01:29:00.109.fits
/data/raw/2005-07-03/WFI.2005-07-04T01:36:38.283.fits
/data/raw/2005-07-03/WFI.2005-07-04T01:44:04.817.fits
/data/raw/2005-07-03/WFI.2005-07-04T03:08:30.113.fits
/data/raw/2005-07-03/WFI.2005-07-04T03:15:39.525.fits
/data/raw/2005-07-03/WFI.2005-07-04T03:23:11.508.fits
/data/raw/2005-07-03/WFI.2005-07-04T03:31:07.894.fits
/data/raw/2005-07-03/WFI.2005-07-04T04:15:16.351.fits
/data/raw/2005-07-03/WFI.2005-07-04T04:22:11.592.fits
```

Then execute the command

```
$mefpp --bias WFI_MB_1x1.fits --mff WFI_SKYFF_BB_Rc_162_ESO844_1x1.fits \
          --science tempel1_Rc.list
```

After some time the program will stop having writen to disk, in **workdir**, the files

WFI.2005-07-03T23:07:50.699_pre.fits
WFI.2005-07-03T23:16:54.380_pre.fits
WFI.2005-07-03T23:23:58.061_pre.fits
WFI.2005-07-03T23:38:16.955_pre.fits
WFI.2005-07-03T23:45:17.787_pre.fits
WFI.2005-07-03T23:52:22.338_pre.fits
WFI.2005-07-04T00:01:44.400_pre.fits
WFI.2005-07-04T00:09:40.275_pre.fits
WFI.2005-07-04T00:17:19.940_pre.fits
WFI.2005-07-04T00:24:33.131_pre.fits
WFI.2005-07-04T00:38:49.766_pre.fits
WFI.2005-07-04T00:47:10.873_pre.fits
WFI.2005-07-04T00:54:27.265_pre.fits
WFI.2005-07-04T01:12:37.306_pre.fits
WFI.2005-07-04T01:19:45.878_pre.fits
WFI.2005-07-04T01:29:00.109_pre.fits
WFI.2005-07-04T01:36:38.283_pre.fits
WFI.2005-07-04T01:44:04.817_pre.fits
WFI.2005-07-04T03:08:30.113_pre.fits
WFI.2005-07-04T03:15:39.525_pre.fits
WFI.2005-07-04T03:23:11.508_pre.fits
WFI.2005-07-04T03:31:07.894_pre.fits
WFI.2005-07-04T04:15:16.351_pre.fits
WFI.2005-07-04T04:22:11.592_pre.fits

These are MEF files 268Mbytes each. They have their CRVALi and CRPIXi keywords corrected, overscan subtracted, bias subtracted, and flat fielded. The overcan is subtracted line by line. We have chosen above to use the sky flat field instead of the master flat field because dome flats for WFI are not very good.

Step 4: The following processing is meant to be used for the mosaicing of observations of a moving object. It uses the python script splitChips.py and of the IRAF command **imcombine**.

Suppose that the above science frames correspond to the observations of comet P/Temple1. Then the user should determine the position of the centroid of the nuclei by hand and write them in a file named cometCentroids.list:

$cat cometCentroids.list
WFI.2005-07-03T23:07:50.699_pre.fits 1047.8 1681.6
WFI.2005-07-03T23:16:54.380_pre.fits 1037.6 1760.2
WFI.2005-07-03T23:23:58.061_pre.fits 1024.9 1588.2
WFI.2005-07-03T23:38:16.955_pre.fits 1005.2 1577.5
WFI.2005-07-03T23:45:17.787_pre.fits 993.4 1655.5
WFI.2005-07-03T23:52:22.338_pre.fits 986.4 1480.4
WFI.2005-07-04T00:01:44.400_pre.fits 1048.7 1680.8
WFI.2005-07-04T00:09:40.275_pre.fits 1019.2 1578.3
WFI.2005-07-04T00:17:19.940_pre.fits 1011.5 1657.4
..... 15 more lines .....

Notice that there are as many lines as MEF files to be combined. The combination

proceeds in two stages. First, the MEF files are split into their 8 extensions and at the same time two files are writen: imageNames.list contains the name of the file in which each extension was saved, and imageOffsets.list containing the offsets to be aplied to each image to produce the mosaic. This is done with the splitChips.py Python script which is installed in the w2p2off machine. This script needs the presence of a few shell scripts which execute IRAF commands from the command line.

$splitChips.py cometCentroids.list

The files write to disk should look something like what follows

$cat imageNames.list

WFI.2005-07-03T23:07:50.699_pre_1.fits
WFI.2005-07-03T23:07:50.699_pre_2.fits
WFI.2005-07-03T23:07:50.699_pre_3.fits
WFI.2005-07-03T23:07:50.699_pre_4.fits
WFI.2005-07-03T23:07:50.699_pre_5.fits
WFI.2005-07-03T23:07:50.699_pre_6.fits
WFI.2005-07-03T23:07:50.699_pre_7.fits
WFI.2005-07-03T23:07:50.699_pre_8.fits
WFI.2005-07-03T23:16:54.380_pre_1.fits
WFI.2005-07-03T23:16:54.380_pre_2.fits
WFI.2005-07-03T23:16:54.380_pre_3.fits
WFI.2005-07-03T23:16:54.380_pre_4.fits
WFI.2005-07-03T23:16:54.380_pre_5.fits
WFI.2005-07-03T23:16:54.380_pre_6.fits
WFI.2005-07-03T23:16:54.380_pre_7.fits
WFI.2005-07-03T23:16:54.380_pre_8.fits
..... 176 more lines .....

$cat imageOffsets.list

-4098.000000 240.000000
-1955.000000 240.000000
187.000000 240.000000
2329.000000 240.000000
2327.000000 -3948.000000
185.000000 -3948.000000
-1957.000000 -3948.000000
-4098.000000 -3948.000000
-4087.800000 161.400000
-1944.800000 161.400000
197.200000 161.400000
2339.200000 161.400000
2337.200000 -4026.600000
195.200000 -4026.600000
-1946.800000 -4026.600000
-4087.800000 -4026.600000
..... 176 more lines

Step 5: Now we are ready to run imcombine from the iraf command line:

```
   NOAO PC-IRAF Revision 2.12.1-EXPORT Fri Jul 12 15:54:09 MST 2002
   This is the EXPORT version of PC-IRAF V2.12 supporting most PC systems.

   Welcome to IRAF.  To list the available commands, type ? or ??.  To get
   detailed information about a command, type `help command'.  To run a
   command or load a package, type its name.  Type `bye' to exit a
   package, or `logout' to get out of the CL.  Type `news' to find out
   what is new in the version of the system you are using.  The following
   commands or packages are currently defined:

     apropos    dimsum.    guiapps.   mscred.    proto.     tables.
     color.     eis.       ifocas.    mxtools.   rvsao.     utilities.
     crutil.    esowfi.    images.    nmisc.     softools.  xccdred.
     ctio.      fitsutil.  immatchx.  noao.      stecf.     xdimsum.
     dataio.    gemini.    language.  obsolete.  stsdas.
     dbms.      gmisc.     lists.     plot.      system.
cl> lpar imcombine
       input = "@imageNames.list" List of images to combine
      output = "tempRc_04Jul.fits" List of output images
    (headers = "")            List of header files (optional)
    (bpmasks = "")            List of bad pixel masks (optional)
   (rejmasks = "")            List of rejection masks (optional)
  (nrejmasks = "")            List of number rejected masks (optional)
   (expmasks = "")            List of exposure masks (optional)
     (sigmas = "")            List of sigma images (optional)
    (logfile = "STDOUT")      Log file\n
    (combine = "median")      Type of combine operation
     (reject = "minmax")      Type of rejection
    (project = no)            Project highest dimension of input images?
    (outtype = "real")        Output image pixel datatype
  (outlimits = "")            Output limits (x1 x2 y1 y2 ...)
    (offsets = "imageOffsets.list") Input image offsets
   (masktype = "none")        Mask type
  (maskvalue = 0.)            Mask value
      (blank = 0.)            Value if there are no pixels\n
      (scale = "none")        Image scaling
       (zero = "median")      Image zero point offset
     (weight = "none")        Image weights
    (statsec = "")            Image section for computing statistics
    (expname = "")            Image header exposure time keyword\n
 (lthreshold = INDEF)         Lower threshold
 (hthreshold = INDEF)         Upper threshold
       (nlow = 1)             minmax: Number of low pixels to reject
      (nhigh = 3)             minmax: Number of high pixels to reject
      (nkeep = 1)             Minimum to keep (pos) or maximum to reject (neg
      (mclip = yes)           Use median in sigma clipping algorithms?
     (lsigma = 3.)            Lower sigma clipping factor
     (hsigma = 3.)            Upper sigma clipping factor
    (rdnoise = "0.")          ccdclip: CCD readout noise (electrons)
       (gain = "1.")          ccdclip: CCD gain (electrons/DN)
     (snoise = "0.")          ccdclip: Sensitivity noise (fraction)
   (sigscale = 0.1)           Tolerance for sigma clipping scaling correction
      (pclip = -0.5)          pclip: Percentile clipping parameter
       (grow = 0.)            Radius (pixels) for neighbor rejection
       (mode = "ql")
cl>
```

README

Wide Field Imager Reduction recipes

(F. Selman, La Silla 17 February 2005)

The recipes found in this package use the originals written by N. Devillard as starting point.

They have been extensively modified to work with WFI MEF files directly, both as input and output. I decided to keep the images allways within a MEF file which is their logical place, at least in the context of a primary processing. For example, the flat field routines now keep track of the median in each of the extension and save the value in the respective BSCALE keywords. This save us from the need to do a gain harmonization of the chips later on, as the flat field give us that directly if we take the precaution of normalizing the flat field frames with a single number for all the extensions. Thus, the user should work in ADUs until after the frames have been flat fielded.

The basic modus operandi is to go extension by extension applying the already proven eclipse routines.

Version 1.0 of these recipes should deal correctly with the following issues:

- Input and output to MEF files
- Proper treatment of overscan correction, subtracting a line rather than a single number, as the overscan varies with y.
- Proper treatment of the BSCALE and BZERO keywords.
- Proper treatment of the CRVALi, and CRPIXi keywords, specially after trimming.

These recipes are intended as efficient and simple to use, voiding the complexities of a pipeline.

These recipes, in the version 1.0 should allow
the users to do the initial pre-processing of
their data correctly.

Version 2.0 should handle correctly the following:

TBD: - BPM creation, and inclusion in the processing
- Creation of a wfi_ccube version to manipulate MEF files.
- Creation of a wfi_peak version to extract info on objects in the images
- Determination of WCS and inclution in headers

Version 3.0 should be general of any instrument
producing MEF files.

PATCHES:

> The current distribution includes a patch
> to the eclipse routine src/unix/comm.c.
> This routine has calls to vafprintf which
> core dumps when run with gcc in ppc. This
> patch is not needed in x86 Linux or in
> Mac OS X. It only bombs in Linux PPC.

F. Selman

README FROM ORIGINAL VERSION:

The recipes found in this directory have been written in the context of the BeoWFI project, aimed at providing a data reduction facility for the WFI instrument based on a Beowulf cluster.

The recipe functionalities are drawn from the eclipse library. The recipe code has been written by N. Devillard, based on inputs from Aniello Grado, Andreas Wicenec, Stefan Bogun, Miguel Albrecht, Peter Quinn. The Beowulf demonstration cluster at ESO is maintained by Dieter Suchar and Andreas Wicenec.

N. Devillard
Wed Oct 25 13:14:02 CEST 2000

TODO (and never done...)
(written 17 February 2005)

-Reading from a PAF file recipe parameters,
and instrument parameters.

-Proper identification of recipes:

    REC MEF BIAS
    REC MEF BPM
    REC MEF OVERSCAN
    REC MEF DOMEFF
    REC MEF SKYFF
    REC MEF MASTERFF

-Clean up: using qfits as far as possible

-Proper handling of writing to files:
see the pixdump example and modify/integrate
into a routine to dump headers and pixels
to MEF files.

-Further separation into header creation and
file writing

-Find proper way to deal with command line options,
and decide on weather we will have a single large
pipe-like processing, or many atomic units.

-Add option to use a "normalized" flat, that is,
a flat that only takes out the pixel-to-pixel
variations.

-Write man page

-Test parameterers for BPM creation

-Test Flat field creation and write script
to monitor stability of flatfield for
keyband pixels:

    Dome/Dome
    Sky/Sky
    Dome/Sky

- Add a linearity correction section

- Add a shutter error correction section
- Add proper WCS keyword handling
- Add registering/warping routine
- Create a few coulour pictures
- Test with OmegaCAM data.